

Caso di studio del corso di Ingegneria della Conoscenza

Cinelogic

Sistema intelligente di raccomandazione cinematografica

Gruppo di lavoro:

- Abderrahmane Cherchari, 775262, a.cherchari@studenti.uniba.it
- Giuseppe Pio Impagnatiello, 776480, g.impagnatiello2@studenti.uniba.it

Link progetto GitHub: <https://github.com/ramensonoio/ProgettoICon25-26.git>

AA 2025-26

Indice

1.	Introduzione	3
1.1.	Dataset utilizzati e librerie	4
1.2.	Pre-processing del dataset	4
1.3.	Gestione delle celle 'null' per ogni colonna	6
1.4.	Rappresentazioni Grafiche	6
1.5.	Individuazione di eventuali correlazioni.....	8
2.	Recommender System	11
2.1.	Sommario	11
2.2.	Scelta della metrica	11
2.3.	Realizzazione	12
2.4.	Classificazione.....	14
2.5.	Modello Adottato	15
2.6.	Dataset.....	16
2.7.	Pre-processing dei dati	16
2.8.	Miglioramento del modello e Hyperparameters Tuning.....	17
2.9.	Predizione su nuovi dati	19
3.	Knowledge Base	21
3.1.	Introduzione	21
3.2.	Gestione della KB.....	21
3.3.	Fatti.....	21
3.4.	Regole	22
3.4.1.	Trova film in base a determinate caratteristiche	23
3.4.2.	Sulla base delle preferenze dell'utente, trova la piattaforma migliore per i film selezionati	26
4.	Ontologia.....	29
4.1.	Sommario	29
4.2.	Protégé	29
4.3.	Owlready2	34
5.	Conclusioni	41
6.	Bibliografia	41

1. Introduzione

Negli ultimi anni i servizi di streaming di film e serie televisive sono diventati una componente centrale dell'intrattenimento digitale. Le piattaforme dedicate alla distribuzione di contenuti audiovisivi rappresentano oggi uno degli strumenti più utilizzati per l'accesso a film e serie TV, offrendo cataloghi sempre più vasti e diversificati.

L'abbondanza di contenuti disponibili, unita alla crescente competizione tra le diverse piattaforme, rende tuttavia complesso per gli utenti orientarsi nella scelta del titolo da visualizzare o della piattaforma più adatta alle proprie esigenze. In questo contesto emerge la necessità di strumenti intelligenti in grado di analizzare le preferenze degli utenti e supportarli nelle loro decisioni.

Considerato l'ampio utilizzo di tali servizi e in quanto noi stessi fruitori delle piattaforme di streaming, abbiamo deciso di sviluppare un sistema che potesse fornire un supporto concreto nella selezione di contenuti audiovisivi e nell'individuazione della piattaforma più conveniente o coerente con i propri gusti.

A tal fine, il progetto si articola nei seguenti componenti:

- Un Recommender System, progettato per suggerire film e serie TV sulla base delle preferenze personali dell'utente. Data l'elevata quantità di contenuti disponibili, la scelta può risultare difficoltosa; il sistema implementato utilizza algoritmi di analisi dei dati per individuare pattern nelle preferenze e proporre contenuti coerenti con i gusti individuali.
- Un modello di classificazione, in grado di prevedere la categoria dei rating assegnati dagli utenti ai contenuti. Questo modulo consente di comprendere meglio le dinamiche di valutazione dei film e delle serie TV e di migliorare ulteriormente la qualità dei suggerimenti forniti dal sistema.
- Una Knowledge Base interattiva, che permette agli utenti di interrogare il sistema per ottenere informazioni dettagliate su film, serie TV e relative caratteristiche.
- Un'ontologia di dominio, finalizzata a rappresentare formalmente il dominio delle piattaforme di streaming, definendo concetti, relazioni e significati dei simboli utilizzati all'interno del sistema.

Una volta avviato il programma, l'utente può interagire dinamicamente con ciascuna componente, scegliendo l'opzione più adeguata alle proprie necessità e passando agevolmente dal Recommender System alla Knowledge Base o all'Ontologia, in un ambiente integrato e coerente.

```
==== Benvenuti in CineLogic ====
```

```
Seleziona un'operazione:
```

- 1) Recommender System
- 2) Interagisci con la KB
- 3) Interagisci con l'ontologia
- 4) Esci

```
Inserisci un valore (1-4):
```

1.1. Dataset utilizzati e librerie

Per realizzare questo sistema abbiamo utilizzato il seguente dataset kaggle:

<https://www.kaggle.com/datasets/farhantanveerhasan/movie-stream-df>

Sono state applicate una serie di operazioni di *pre-processing*, descritte in seguito.

Tutto il lavoro è stato svolto in linguaggio Python sfruttando i code editor **VScode** e **Jupyter**.

Nello sviluppo del progetto, si è fatto largo uso delle librerie **Pandas** e **NumPy**.

1.2. Pre-processing del dataset

Dopo aver osservato gli elementi che costituiscono il dataset, passiamo alla fase di *pre-processing* del dataset.

La prima operazione che abbiamo eseguito è la rimozione delle colonne che non sono utili ai nostri scopi quali:

- **type**: in quanto il dataset fa riferimento esclusivamente a film, non è utile la distinzione per tipo
- **Season**: dato che gli elementi del dataset sono univocamente film
- **imdb_id**: poiché non è necessaria l'identificazione univoca del titolo sulla piattaforma imdb
- **genres**: abbiamo pensato che la ricerca del titolo debba avvenire solo in base al genere che maggiormente rispecchia il titolo indicato.
- **ncost**: identificativo che non è utile al nostro scopo
- **primaryName**: la ricerca del titolo in base ad un attore che vi è presente debba avvenire tra gli elementi dell'insieme 'name' e non basarsi sull'attore protagonista

```
In [9]: columns_to_drop = ['type', 'seasons', 'imdb_id', 'genres', 'budget', 'ncost', 'primaryName']
movies = movies.drop(columns=columns_to_drop, errors='ignore')
movies.head()
```

Out[9]:

	Unnamed: 0	id	title	description	release_year	runtime	production_countries	imdb_score	imdb_votes	tmdb_popularity	tmdb_score	streaming
0	0	tm1000001	The Lucky Texan	Jerry Mason, a young Texan, and Jake Benson, a...	1934	61	US	5.6	1213.0	4.1	4.7	
1	1	tm1000022	Boonie Bears: The Wild Life	Bear brothers Briar and Bramble set off on an ...	2021	99	CN	5.4	117.0	6.1	3.8	
2	2	tm1000169	Bad Cupid	Archie is a God on a mission to ensure that tr...	2021	81	US	4.4	181.0	3.3	4.8	
3	3	tm1000186	Carol's Christmas	Scrooge encounters the ghost of her late busin...	2021	70	US	2.0	48.0	0.6	6.0	
4	4	tm1000203	Digging to Death	David Van Owen moves into a mysterious house a...	2021	96	US	4.5	464.0	3.5	5.8	

Dall'analisi del dataset abbiamo notato che le metriche "imdb_score", "tmdb_popularity", "tmdb_score", che rappresentano rispettivamente le valutazioni dei titoli delle piattaforme "imdb" e "tmdb", presentano alcuni valori che non vengono approssimati correttamente.

In particolare, le metriche “imdb_score” e “tmdb_score” presentano alcuni valori che non vengono approssimati alla prima cifra decimale.

Per cui procediamo ad approssimarle correttamente:

```
In [8]: df = pd.DataFrame(movies)
df['imdb_score'] = df['imdb_score'].round(1)
df['tmdb_score'] = df['tmdb_score'].round(1)
print(df[['imdb_score', 'tmdb_score']].to_string(index=False))
df.to_csv('pre_processed_dataset.csv', index=False)
```

imdb_score	tmdb_score
5.6	4.7
5.4	3.8
4.4	4.8
2.0	6.0
4.5	5.8
7.5	7.4
4.0	8.0
6.9	5.4
6.9	6.3
4.5	5.0
5.1	6.0
4.7	5.6
4.1	4.4
8.0	8.2
4.0	4.5
5.7	6.0
3.5	5.1
6.1	6.0
6.3	6.3

Procediamo con il rinominare le due colonne “main_genre” e “name” in “genre” e “actors” in modo che il loro scopo sia più chiaro.

```
In [9]: df = pd.read_csv('pre_processed_dataset.csv')
df.rename(columns={'main_genre': 'genre', 'name': 'actors'}, inplace=True)
df.to_csv('preProcessed_dataset.csv', index=False)
```

Rinominare la colonna “subscription_cost” in “monthly_subscription_cost” e convertiamo i valori di questa colonna in float con due cifre decimali.

```
In [10]: df = pd.read_csv('preProcessed_dataset.csv')
df.rename(columns={'subscription_cost': 'monthly_subscription_cost'}, inplace=True)
df['monthly_subscription_cost'] = df['monthly_subscription_cost'].replace(to_replace=r'\D', value='', regex=True).astype(float)
df['monthly_subscription_cost'] = df['monthly_subscription_cost'] / 100
df.to_csv('pre_processed_dataset.csv', index=False)
print(df['monthly_subscription_cost'])
0      14.99
1      14.99
2      14.99
3      14.99
4      14.99
...
18714    4.99
18715    4.99
18716    4.99
18717    4.99
18718    4.99
Name: monthly_subscription_cost, Length: 18719, dtype: float64
```

1.3. Gestione delle celle 'null' per ogni colonna

```
In [14]: movies.isnull().sum()
```

```
Out[14]: Unnamed: 0      0
id          0
title       0
description  0
release_year 0
runtime     0
production_countries 0
imdb_score  0
imdb_votes  0
tmdb_popularity 0
tmdb_score  0
streaming_service 0
main_genre  0
name        0
monthly_subscription_cost 0
dtype: int64
```

Da questa funzione risulta che il nostro dataset non presenta alcun valore da gestire.

1.4. Rappresentazioni Grafiche

Istogramma che rappresenta la distribuzione dei titoli cinematografici tra le varie piattaforme di streaming presenti nel dataset:

```
In [15]: df = pd.DataFrame(movies)
```

```
streaming_counts = df['streaming_service'].value_counts()
```

```
#creazione dell'istogramma
```

```
plt.figure(figsize=(10, 6))
```

```
streaming_counts.plot(kind='bar', color='skyblue')
```

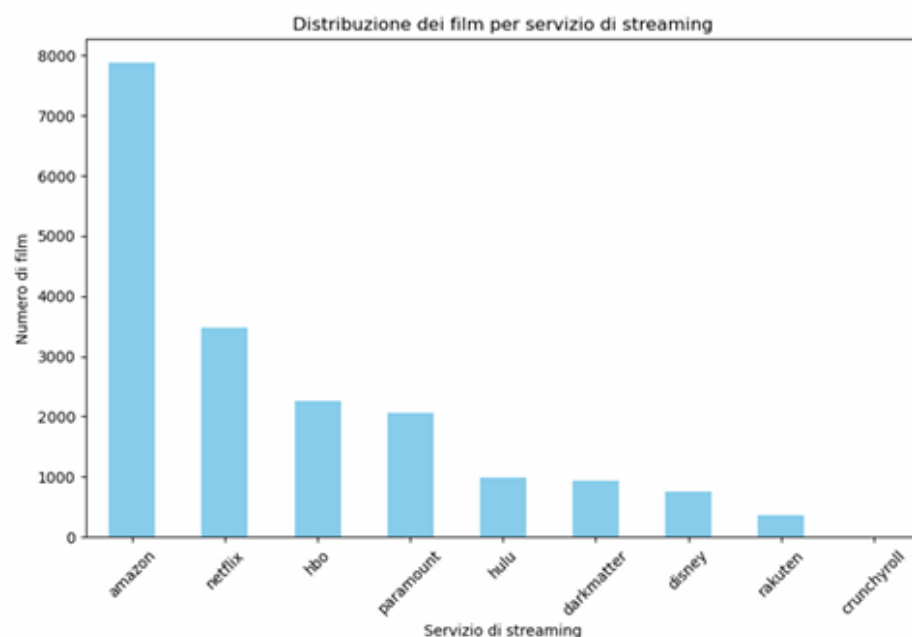
```
plt.title('Distribuzione dei film per servizio di streaming')
```

```
plt.xlabel('Servizio di streaming')
```

```
plt.ylabel('Numero di film')
```

```
plt.xticks(rotation=45)
```

```
Out[15]: (array([0, 1, 2, 3, 4, 5, 6, 7, 8]),
[Text(0, 0, 'amazon'),
Text(1, 0, 'netflix'),
Text(2, 0, 'hbo'),
Text(3, 0, 'paramount'),
Text(4, 0, 'hulu'),
Text(5, 0, 'darkmatter'),
Text(6, 0, 'disney'),
Text(7, 0, 'rakuten'),
Text(8, 0, 'crunchyroll')])
```



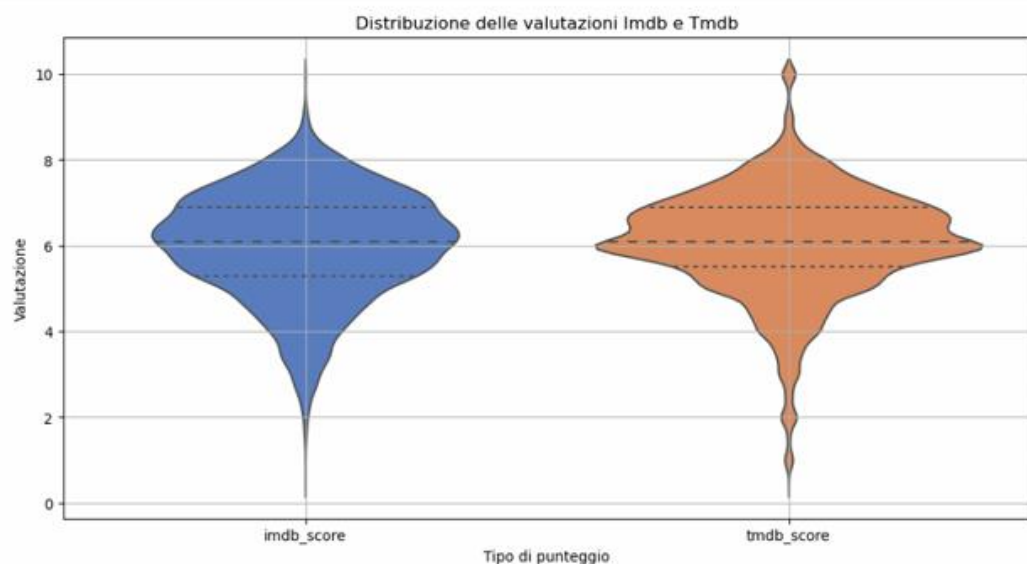
Dal grafico realizzato possiamo cogliere visivamente la distribuzione dei titoli tra le varie piattaforme di streaming che sono state considerate all'interno del dataset. Questi dati ci fanno rendere conto che la maggior parte dei titoli sono detenuti dalle piattaforme "Amazon" e "Netflix", per cui le raccomandazioni saranno, con probabilità più elevate delle altre, indirizzate verso queste ultime.

Grafico a violino che mostra la distribuzione delle valutazioni imdb_score e tmdb_score:

```
In [16]: df = pd.DataFrame(movies)

#convertiamo i dati in un formato 'long' adatto per seaborn
df_long = pd.melt(df, id_vars=['title'], value_vars=['imdb_score', 'tmdb_score'], var_name='score_type')
#creazione del grafico a violino
plt.figure(figsize=(12, 6))
sns.violinplot(x='score_type', y='score', data=df_long, inner='quartile', palette='muted')
plt.title('Distribuzione delle valutazioni Imdb e Tmdb')
plt.xlabel('Tipo di punteggio')
plt.ylabel('Valutazione')
plt.grid(True)

plt.show()
```



Dal grafico abbiamo rilevato una distribuzione delle votazioni per ciascun livello di valutazione. Ciascun violino rappresenta la variazione delle valutazioni assegnate dalle due piattaforme (imdb e tmdb) rispetto a ciascun titolo.

Da questi due grafici possiamo notare che la maggior parte delle valutazioni assegnate assumono un valore vicino al 6.

Istogramma che rappresenta la distribuzione dei prezzi tra le varie piattaforme di streaming:

```
In [17]: df = pd.DataFrame(movies)

streaming_services=['Amazon', 'Netflix', 'Disney', 'Hulu', 'Hbo', 'Darkmatter', 'Paramo

plt.figure(figsize=(10, 6))
hist = sns.histplot(data=df, x='monthly_subscription_cost', hue='streaming_service', mu
plt.title('Distribuzione dei costi di abbonamento alle piattaforme di streaming')
plt.xlabel('prezzo')
plt.ylabel('conteggio')
plt.grid(True)

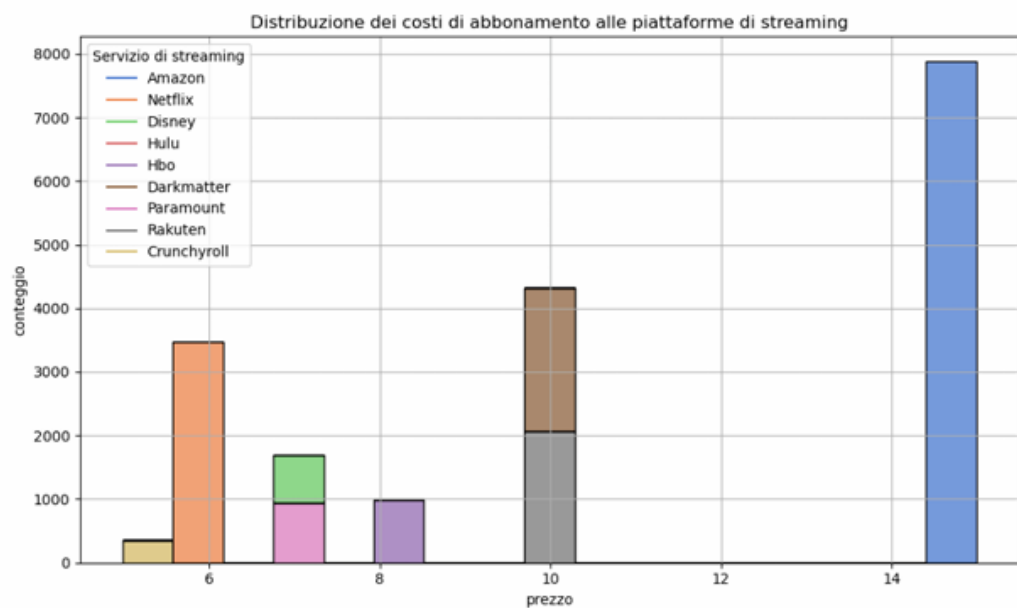
handles, labels = hist.get_legend_handles_labels()
if not handles:
    #recuperiamo i colori della palette usata da seaborn
    palette = sns.color_palette('muted', len(streaming_services))
    color_dict = dict(zip(streaming_services, palette))

    #creiamo i gestori manualmente con i colori corretti
    for service in streaming_services:
        handles.append(plt.Line2D([], [], color=color_dict[service], label=service))

plt.legend(handles=handles, title='Servizio di streaming')

plt.tight_layout()
plt.show()

/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: FutureWarning: u
se_inf_as_na option is deprecated and will be removed in a future version. Convert inf
values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



Dall'istogramma realizzato possiamo cogliere la diversa distribuzione dei prezzi degli abbonamenti messi a disposizione delle varie piattaforme di streaming considerate all'interno del dataset.

1.5. Individuazione di eventuali correlazioni

Andiamo a sfruttare la seguente funzione per visualizzare il numero dei film messi a disposizione da ciascuna piattaforma di streaming presa in considerazione all'interno del dataset, con lo scopo di analizzare la correlazione tra il numero di film presente in ciascuna piattaforma e i generi maggiormente presenti in queste ultime.


```
In [18]: df = pd.DataFrame(movies)

film_count_per_service = df.groupby('streaming_service').size().reset_index(name='number_of_movies')
print(film_count_per_service)
```

	streaming_service	number_of_movies
0	amazon	7875
1	crunchyroll	1
2	darkmatter	940
3	disney	758
4	hbo	2264
5	hulu	987
6	netflix	3477
7	paramount	2062
8	rakuten	355

Per ciascuna piattaforma di streaming, andiamo ad analizzare la distribuzione dei generi dei vari titoli presenti all'interno delle loro raccolte.

```
In [25]: import math

df = pd.DataFrame(movies)

streaming_services = df['streaming_service'].unique()

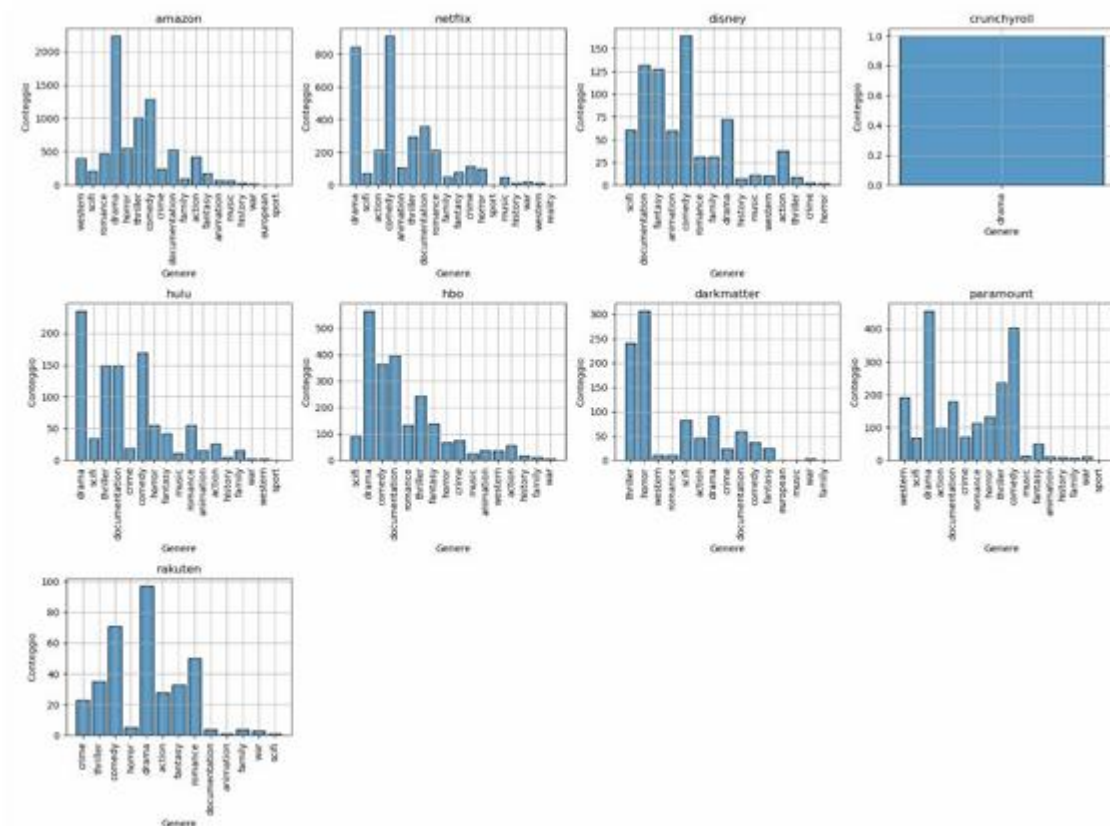
df.replace([float('inf'), -float('inf')], pd.NA, inplace=True)

num_services = len(streaming_services)
num_cols = 4
num_rows = math.ceil(num_services / num_cols)

plt.figure(figsize=(16, num_rows * 4))
for i, service in enumerate(streaming_services, 1):
    plt.subplot(num_rows, num_cols, i)
    subset = df[df['streaming_service'] == service]
    sns.histplot(data=subset, x='main_genre', discrete=True, shrink=0.8)

    plt.title(f'{service}')
    plt.xlabel('Genere')
    plt.ylabel('Conteggio')
    plt.xticks(rotation=90)
    plt.grid(True)

plt.tight_layout()
plt.show()
```



Infine, restituiamo il conteggio dei generi principali per ciascuna piattaforma di streaming presa in considerazione.

```
In [5]: df = pd.DataFrame(movies)
genre_counts = df.groupby(['streaming_service', 'main_genre']).size().reset_index(name='count')
print(genre_counts)
```

	streaming_service	main_genre	count
0	amazon	action	419
1	amazon	animation	78
2	amazon	comedy	1289
3	amazon	crime	244
4	amazon	documentation	535
..	..	..	..
125	rakuten	horror	5
126	rakuten	romance	50
127	rakuten	scifi	1
128	rakuten	thriller	35
129	rakuten	war	3

[130 rows x 3 columns]

2. Recommender System

2.1. Sommario

Un sistema di raccomandazione, o motore di raccomandazione, è un software progettato per filtrare i contenuti e fornire suggerimenti personalizzati agli utenti, facilitando così le loro decisioni. Ci sono tre principali modalità di creazione delle raccomandazioni: l'approccio collaborativo, l'approccio basato sui contenuti e l'approccio ibrido.

Abbiamo scelto di adottare l'approccio del Content-based Filtering, in quanto il dataset selezionato ci ha permesso di accedere ad una vasta gamma di informazioni dettagliate sui film, organizzate in categorie. Questo metodo si basa sull'analisi del contenuto di un elemento e sul confronto con il profilo dell'utente. Il contenuto di un elemento include la sua descrizione, i suoi attributi, le parole chiave e le etichette associate. Questi dati vengono confrontati con il profilo dell'utente, che riflette le sue preferenze.

Nel nostro caso specifico, non disponendo di un profilo utente predefinito, abbiamo optato per un'interazione diretta con l'utente. Quest'ultimo inserisce i dati relativi al film che ha scelto o che preferisce, e riceve una raccomandazione personalizzata basata su tali informazioni.

```
=== BENVENUTO NEL RECOMMENDER SYSTEM ===  
  
Digita le caratteristiche del film  
su cui vuoi che si avvii la raccomandazione.  
--- Ricerca Film ---  
Inserisci i dati del film che ti interessa:  
  
Inserisci il titolo: |
```

2.2. Scelta della metrica

Prima di procedere con lo sviluppo del Recommender System, ci siamo dedicati allo studio delle metriche per il calcolo delle similarità, optando per la **Similarità del Coseno** (*Cosine Similarity*).

La Similarità del Coseno è una metrica matematica impiegata per determinare quanto due vettori siano simili, misurando il coseno dell'angolo compreso tra di essi all'interno di uno spazio multi-dimensionale. A differenza di altre misure statistiche (come la Correlazione di Pearson o la Distanza Euclidea), questa metrica valuta esclusivamente l'orientamento e la direzione logica dei vettori, ignorandone la magnitudo (ossia la lunghezza fisica del vettore).

In questo contesto, essa consente di riconoscere l'affinità tematica e semantica tra due film indipendentemente dalla lunghezza delle stringhe di testo analizzate. Poiché le matrici TF-IDF non contengono valori negativi, l'indice calcolato assumerà un valore compreso tra 0 e 1. Un valore pari a 0 indica vettori perfettamente ortogonali (nessuna similarità), mentre un valore pari a 1 indica vettori perfettamente allineati e proporzionali (similarità semantica totale).

2.3. Realizzazione

Per realizzare il Recommender System basato sul **Content-Based Filtering**, abbiamo fatto uso della libreria open source Python scikit-learn, che ci ha permesso di utilizzare strumenti avanzati dedicati al *Natural Language Processing* (NLP).

Per calcolare le raccomandazioni, abbiamo aggregato le informazioni testuali e descrittive più rilevanti del dataset (nello specifico le feature: "title", "release_year", "runtime", "production_countries" e "genre"), concatenandole in un'unica stringa di testo per ogni film. Poiché i dati in questo formato non sono direttamente elaborabili per calcolare distanze matematiche, è stato necessario trasformarli in vettori numerici. Utilizzando la tecnica **TF-IDF (Term Frequency-Inverse Document Frequency)**, questi testi vengono convertiti generando una matrice sparsa in cui ogni riga rappresenta un film e ogni colonna rappresenta una specifica parola (o "token") appartenente al vocabolario globale del dataset. In sostanza, il punteggio TF-IDF misura la frequenza di una parola in un documento, ma la pondera inversamente in base al numero di documenti in cui quella parola compare. Questo approccio riduce l'impatto di parole di uso molto comune, dando maggior peso ai termini specifici che caratterizzano realmente un film.

Per affinare ulteriormente la precisione semantica, tramite l'oggetto `TfidfVectorizer` di scikit-learn, abbiamo filtrato le *stop-words* (parole vuote o articoli grammaticali) e abilitato l'uso di bi-grammi, permettendo al sistema di riconoscere sia parole singole che coppie di parole associate (es. *Science Fiction*).

```
def vectorize_data(movie_data):  
    # Inizializzo  
    vectorizer = TfidfVectorizer(  
        analyzer='word',  
        stop_words='english', # Ignora "the", "a", "is", ecc.  
        # Legge sia parole singole ("Action") che coppie ("Science Fiction")  
        ngram_range=(1, 2)  
    )  
  
    # Addestramento e trasformazione del testo in matrice matematica  
    tfidf_matrix = vectorizer.fit_transform(movie_data['all_content'])  
  
    # Restituisco la matrice sparsa  
    return tfidf_matrix
```

Una volta ottenuta la matrice TF-IDF, siamo passati al calcolo delle similarità sostituendo i classici indici statistici con la **Similarità del Coseno (Cosine Similarity)**. A differenza della Correlazione di Pearson, che valuta la covarianza ed è computazionalmente pesante su matrici ad altissima dimensionalità, la Similarità del Coseno misura il coseno dell'angolo tra due vettori proiettati nello spazio multi-dimensionale, ignorandone la lunghezza e concentrandosi puramente sull'orientamento semantico. Questo ci ha permesso di sfruttare l'algebra matriciale vettorializzata, rendendo il calcolo estremamente efficiente.

```

# Vettorizzazione
tfidf_matrix = vectorize_data(movie_data)

print("\nInizio calcolo delle similarità...")

# Calcolo delle Similarità
# Estraiamo il vettore del film target
target_vector = tfidf_matrix[target_index]

# cosine_similarity calcola istantaneamente la distanza tra il target e T
# Restituisce un array di punteggi (da 0 a 1)
similarities = cosine_similarity(target_vector, tfidf_matrix).flatten()

# Estrazione dei Top 5
# enumerate abbina l'indice di riga al punteggio: [(0, score), (1, score)]
corr_list = list(enumerate(similarities))

# Ordiniamo per punteggio (x[1]) decrescente.
# Prendiamo [1:6] per saltare il primo risultato (il film stesso, score=1)
sorted_corr = sorted(corr_list, key=lambda x: x[1], reverse=True)[1:6]

# Estraiamo solo gli indici
movie_index = [item[0] for item in sorted_corr]

```

All'atto pratico, quando l'utente richiede una raccomandazione, il sistema effettua un *reverse mapping* per individuare l'indice del film richiesto all'interno del dataset. Successivamente, estrae il vettore TF-IDF corrispondente a quel film e calcola istantaneamente il punteggio di Similarità del Coseno rispetto all'intera matrice dei film rimanenti.

Il sistema ordina infine questi punteggi in modo decrescente, restituendo all'utente i primi 5 film che presentano il grado di similarità semantica più elevato.

```
Modello ottimizzato pronto. Ora possiamo procedere alla fase di recommendation...

Ecco una lista di 5 film più simili a quello indicato, con una predizione sulla categoria star:
```

	title	release_year	genre	streaming_service	star	\
4942	Antariksham 9000 KMPH	2018	scifi	amazon	4	
10110	Qismat	2018	drama	netflix	5	
4833	Loveyatri	2018	comedy	amazon	3	
5129	The Tomorrow War	2021	scifi	amazon	4	
10072	Baazaar	2018	crime	netflix	4	

	star_prediction
4942	4
10110	4
4833	4
5129	4
10072	4

2.4. Classificazione

Oltre al Recommender System, abbiamo introdotto una classificazione degli elementi del dataset basata su una nuova variabile target chiamata “star”. Questa categoria è stata creata ingegnerizzando due feature già esistenti relative al rating dei film: `imdb_score` e `tmdb_score`. Inizialmente, abbiamo calcolato una nuova colonna chiamata `media_voti`, che rappresenta la media aritmetica di questi due campi.

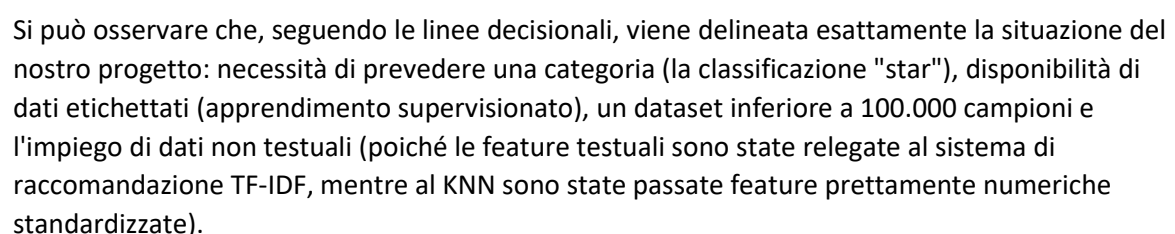
```
#creazione categoria star (media_voti)
movie_data['media_voti'] = (movie_data['imdb_score'] + movie_data['tmdb_score']) / 2

# pd.cut divide i dati in fasce. I bin sono i limiti: [0, 5, 7.5, 10]
# labels sono i valori da assegnare a quelle fasce: [1, 2, 3]
movie_data['star'] = pd.cut(
    movie_data['media_voti'],
    bins=[0, 5.0, 7.5, 10.0],
    labels=[1, 2, 3],
    include_lowest=True
)
```

Per trasformare questo valore numerico continuo in categorie discrete (tecnica nota nel Machine Learning come *Binning* o *Discretizzazione*), ci siamo avvalsi della funzione nativa `pd.cut` della libreria Pandas. In fase di studio dei dati, abbiamo notato che una classificazione a 5 stelle avrebbe generato un dataset fortemente sbilanciato (con pochissimi campioni nelle classi inferiori), il che avrebbe compromesso l'addestramento del modello e reso metriche come la Precisione matematicamente mal definite per alcune categorie. Per risolvere questo problema e ottimizzare le performance predittive, abbiamo deciso di raggruppare i voti in 3 macro-fasce principali, assegnando rispettivamente le etichette 1, 2 e 3. Successivamente, sono state condotte una serie di valutazioni sul modello scelto, seguite da una fase di ottimizzazione (Hyperparameter Tuning). Per quanto riguarda il modello di classificazione, abbiamo optato per il `KNeighborsClassifier` (KNN), disponibile all'interno della libreria Scikit-Learn.

Il K-Nearest Neighbors, o KNN, è un algoritmo impiegato nel riconoscimento di pattern per classificare oggetti in base alle caratteristiche dei loro vicini. In pratica, l'input consiste nei k esempi di addestramento più vicini all'oggetto in questione all'interno dello spazio delle feature (caratteristiche). Sebbene possa essere applicato sia a problemi di regressione che di classificazione, è comunemente utilizzato per quest'ultima, basandosi sull'assunto che oggetti simili tendano ad essere vicini tra loro.

Per selezionare il modello, abbiamo utilizzato la mappa decisionale (Cheat-sheet) disponibile sul sito ufficiale della libreria Scikit-Learn. Questa mappa guida nella ricerca del miglior estimatore di Machine Learning in base agli obiettivi specifici.



Inoltre, per quanto riguarda l'ottimizzazione del modello, invece di affidarci ai parametri di default, abbiamo implementato una fase di ottimizzazione avanzata. Per determinare il valore ottimale di k (numero di vicini), la funzione di peso (uniforme o basata sulla distanza) e la metrica per il calcolo della distanza (Euclidea, Manhattan, ecc.), abbiamo impiegato la tecnica

della RandomizedSearchCV. Questo ci ha permesso di testare svariate combinazioni di iperparametri attraverso una validazione incrociata (Cross-Validation), estraendo infine la configurazione del modello in grado di garantire le migliori performance predittive sui nostri dati.

2.6. Dataset

Per ottenere un dataset adatto all'uso del modello selezionato e per la classificazione, esso è stato diviso in due parti: training e test. La sezione di training verrà utilizzata per addestrare il modello, che sarà poi valutato sulla parte rimanente, ossia il test. Nel nostro caso, abbiamo deciso di riservare l'80% dei dati per il training e il 20% per il test.

Un aspetto fondamentale è l'assegnazione casuale degli elementi del dataset alle due parti, assicurando che entrambe siano rappresentative del dataset originale e ben distribuite. Questo si può ottenere impostando il parametro "random_state" a un numero intero qualsiasi, dato che non è un parametro da ottimizzare.

Infine, poiché alcuni problemi di classificazione presentano categorie con numero di esempi sbilanciato, è preferibile suddividere il dataset in training e test in modo che mantenga la stessa proporzione di esempi per ogni categoria presente nel dataset originale. Questa tecnica è nota come "split stratificato" e può essere implementata modificando il parametro "stratify" utilizzando la componente "y" del dataset originale.

```
#dividiamo il dataset in due parti, 80% destinato al training e 20% destinato al testing
X_train, X_test, y_train, y_test = train_test_split(*arrays: x, y, test_size=0.2, random_state=1, stratify=y)
```

2.7. Pre-processing dei dati

Abbiamo eseguito la scalatura dei dati per uniformare il range di variazione delle caratteristiche del dataset. Questo processo ha contribuito a migliorare la qualità dei risultati finali, poiché la scalatura riduce il tempo necessario all'algoritmo di apprendimento per convergere al risultato finale.

```
# trasformiamo i dati per renderli adeguati
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
predict_data_scaled = scaler.transform(predict_data)
```

Per quanto riguarda la fase di addestramento e ottimizzazione, abbiamo scelto di incapsulare l'intera logica all'interno di una funzione dedicata (compare_and_select_best_model), al fine di mantenere il codice principale modulare e pulito.


```

compare_and_select_best_model(X_train, X_test, y_train, y_test):

print("\n--- COMPOSIZIONE MODELLO BASE ---")
# Uso i parametri di default espliciti per chiarezza
knn_base = KNeighborsClassifier(n_neighbors=5, weights='uniform')
knn_base.fit(X_train, y_train)

# Controllo visivo sui primi 5 elementi
print(f"Predizioni primi 5 elementi: {knn_base.predict(X_test)[0:5]}")
print(f"Valori effettivi: {list(y_test[0:5])}")

print("\nValutazione del modello Base:")
y_pred_base = knn_base.predict(X_test)
pred_prob_base = knn_base.predict_proba(X_test)
evaluate_model(y_test, y_pred_base, pred_prob_base)

# Addestramento
knn_model = compare_and_select_best_model(X_train, X_test, y_train, y_test)

# Previsione sui nuovi dati raccomandati
star_prediction = knn_model.predict(predict_data_scaled)

# Output
pd.set_option('display.max_columns', None)

recommend_data['star_prediction'] = star_prediction

print("\nEcco una lista di 5 film più simili a quello indicato, con una predizione sulla categoria star:\n")
print(recommend_data)
print("\n")

```

2.8. Miglioramento del modello e Hyperparameters Tuning

Dopo aver sperimentato il modello (Baseline) costruito con i parametri predefiniti di Scikit-Learn, abbiamo generato le prime predizioni per la categoria target “star”. Successivamente, abbiamo valutato questo modello iniziale per analizzarne le performance.

```

Composizione iniziale del modello con iper-parametri di base...

Predizioni dei primi 5 elementi: [3. 3. 3. 3. 3.] Valori effettivi: [3. 3. 3. 3. 3.]

Valutazione del modello...

Classification Report:

```

	precision	recall	f1-score	support
1.0	1.00	1.00	1.00	1
2.0	0.78	0.82	0.80	34
3.0	1.00	1.00	1.00	3709
accuracy			1.00	3744
macro avg	0.93	0.94	0.93	3744
weighted avg	1.00	1.00	1.00	3744

```

ROC Score: 0.9994215045119063

```

Per individuare diverse metriche di valutazione (quali Precision, Recall e F1-score) abbiamo utilizzato il “Classification Report” della libreria `sklearn.metrics`. Inoltre, è stato calcolato il ROC AUC Score (Area Under the Receiver Operating Characteristic Curve), ottenendo un valore di base pari a **0.53**. Il ROC AUC Score rappresenta l’area bidimensionale sottesa all’intera curva ROC e fornisce una valutazione complessiva ed estremamente affidabile delle prestazioni di un modello di classificazione su tutte le possibili soglie decisionali.

Con l’obiettivo di incrementare questo punteggio e migliorare le metriche complessive, abbiamo sottoposto il modello a una fase di *Hyperparameter Tuning* (ricerca dei parametri ottimali).

Abbiamo deciso di utilizzare la classe `RandomizedSearchCV`, combinata con una Cross-Validation (validazione incrociata) di tipo `RepeatedKfold`. La *RandomizedSearchCV* è una tecnica che esplora combinazioni casuali di iperparametri per trovare la migliore configurazione possibile. Rispetto a una tradizionale “Grid Search” (che testa ogni singola combinazione esaustivamente), questa tecnica risulta computazionalmente più rapida ed efficiente, riuscendo molto spesso a individuare set di parametri ottimali in una frazione del tempo.

Sono state definite diverse opzioni per i parametri di ricerca, in particolare:

- Il numero di vicini (`n_neighbors`).
- La tipologia di pesi (`weights`: uniform o distance).
- La metrica di calcolo della distanza (`metric`: euclidean, manhattan, hamming).

```

def search_hyperparameters(X_train, y_train):
    # Definisco lo spazio degli iperparametri
    hyperparameters = {
        'n_neighbors': list(range(1, 30)),
        'weights': ['uniform', 'distance'],
        'metric': ['euclidean', 'manhattan', 'hamming']
    }

    # Inizializzo il modello base e la validazione incrociata
    knn = KNeighborsClassifier()
    cv_fold = RepeatedKfold(n_splits=10, n_repeats=3, random_state=1)

    # Imposto la Randomized Search
    random_search = RandomizedSearchCV(
        estimator=knn,
        cv=cv_fold,
        param_distributions=hyperparameters,
        n_iter=15, # Proviamo 15 combinazioni diverse
        random_state=1 # Per riproducibilità
    )

    # Addestro la ricerca sui soli dati di training
    random_search.fit(X_train, y_train)

    # Estraggo e restituisco direttamente il modello vincitore
    return random_search.best_estimator_

```

La ricerca è stata configurata per eseguire 15 iterazioni complessive (n_iter=15). Durante questo processo, l'algoritmo ha testato internamente le varie combinazioni sulle diverse pieghe della validazione incrociata, valutandone le performance in totale autonomia. Al termine dell'esecuzione, la RandomizedSearchCV ha identificato ed estratto nativamente il miglior set di iperparametri in assoluto.

```

--- RICERCA IPERPARAMETRI (RANDOMIZED SEARCH) ---
Best Weights: uniform
Best Metric: manhattan
Best n_neighbors: 28

```

Una volta estratto il modello vincente (best_estimator_), questo è stato utilizzato per generare le predizioni finali sulla categoria “star”. Il processo di ottimizzazione ha avuto esito positivo, portando il ROC AUC Score finale a un valore incrementato pari a **0.59**

2.9. Predizione su nuovi dati

Dopo aver ottimizzato i parametri del nostro modello, abbiamo utilizzato la nuova configurazione per effettuare predizioni sui film consigliati all'utente nella categoria star.

	title	release_year	genre	streaming_service	star	\
4942	Antariksham 9000 KMPH	2018	scifi	amazon	2	
10110	Qismat	2018	drama	netflix	3	
4833	Loveyatri	2018	comedy	amazon	1	
5129	The Tomorrow War	2021	scifi	amazon	2	
10072	Baazaar	2018	crime	netflix	2	
	star_prediction					
4942	2					
10110	2					
4833	2					
5129	2					
10072	2					

Possiamo osservare che, sebbene le predizioni non siano perfette al 100% rispetto ai risultati precedenti, hanno comunque mostrato dei miglioramenti.

3. Knowledge Base

3.1. Introduzione

Una Knowledge Base (KB) è una raccolta organizzata di informazioni e regole che un sistema può impiegare per risolvere problemi, rispondere a domande o prendere decisioni attraverso la conoscenza che contiene.

Essa rappresenta fatti del mondo sui quali applica delle regole, ovvero linee guida che definiscono come tali fatti possono essere utilizzati o combinati per derivare nuova conoscenza. Possiamo dunque definire una KB come un insieme di proposizioni, dette assiomi, che si presumono essere vere e che consentono di creare un ambiente capace di raccogliere, organizzare e diffondere l'informazione.

Nel contesto del nostro caso di studio, la KB viene impiegata per permettere all'utente di formulare query e ottenere risposte relative all'ambito cinematografico di interesse.

3.2. Gestione della KB

Per la gestione della KB abbiamo impiegato l'estensione Pyswip del linguaggio Python, che si basa sul linguaggio di programmazione logica Prolog.

Grazie alle funzionalità offerte da Prolog abbiamo creato e popolato la Knowledge Base con fatti che vengono generati automaticamente prelevando i dati pertinenti dal dataset.

Successivamente, abbiamo implementato le regole basandoci sui fatti presenti nel KB e su come potrebbe avvenire l'interazione con l'utente.

3.3. Fatti

I fatti rappresentano gli assiomi sempre veri della KB e costituiscono la base delle informazioni su cui operano le regole.

- 1) "title (id, title)"
Rappresenta la relazione tra un film e il suo titolo.
- 2) "description (id, description)"
Rappresenta la relazione tra un film e la sua trama.
- 3) "release_year (id, release_year)"
Rappresenta la relazione tra un film e l'anno del suo rilascio.
- 4) "runtime (id, runtime)"
Rappresenta la relazione tra un film e la sua durata.
- 5) "genre (id, genre)"
Rappresenta la relazione tra un film e il suo genere.
- 6) "production_countries (id, production_countries)"
Rappresenta la relazione tra un film e il suo paese di produzione.
- 7) "streaming_service (id, streaming_service)"
Rappresenta la relazione tra un film e la piattaforma di streaming su cui vederlo.
- 8) "actors (id, actors)"
Rappresenta la relazione tra un film e il nome degli attori che vi recitano.
- 9) "monthly_subscription_cost (id, monthly_subscription_cost)"
Rappresenta la relazione tra un film ed il costo di abbonamento mensile della piattaforma per vederlo in streaming.

```

# Direttive-
f.write(":- discontiguous title/2.\n")
f.write(":- discontiguous description/2.\n")
f.write(":- discontiguous release_year/2.\n")
f.write(":- discontiguous runtime/2.\n")
f.write(":- discontiguous genre/2.\n")
f.write(":- discontiguous actors/2.\n")
f.write(":- discontiguous production_countries/2.\n")
f.write(":- discontiguous streaming_service/2.\n")
f.write(":- discontiguous monthly_subscription_cost/2.\n\n")

# Fatti
for index, row in df.iterrows():
    f.write(f"title({row['id']}, '{row['title']}').\n")
    f.write(f"description({row['id']}, '{row['description']}').\n")
    f.write(f"release_year({row['id']}, {int(row['release_year'])}).\n")
    f.write(f"runtime({row['id']}, {int(row['runtime'])}).\n")
    f.write(f"genre({row['id']}, '{row['genre']}').\n")
    f.write(f"actors({row['id']}, '{row['actors']}').\n")
    f.write(f"production_countries({row['id']}, '{row['production_countries']}').\n")
    f.write(f"streaming_service({row['id']}, '{row['streaming_service']}').\n")
    f.write(f"monthly_subscription_cost({row['id']}, {row['monthly_subscription_cost']}').\n")

```

Abbiamo utilizzato la direttiva :- **discontiguous** in quanto le clausole dei predicati definiti all'interno della KB sono organizzate in modo non contiguo nel file, poiché raggruppate in modo contiguo per ogni film e non per predicato.

Esempio di sezione della Knowledge Base:

```

title(100001, 'The Lucky Texan').
description(100001, 'Jerry Mason, a young Texan, and Jake Benson, an old rancher, become partners and strike it
release_year(100001, 1934).
runtime(100001, 61).
genre(100001, 'western').
actors(100001, '[John Wayne, Barbara Sheldon, Lloyd Whitlock, "George Gabby Hayes", Yakima Canutt, Eddie Parker,
production_countries(100001, 'US').
streaming_service(100001, 'amazon').
monthly_subscription_cost(100001, 14.99).

```

3.4. Regole

Le regole rappresentano il metodo attraverso il quale l'utente può interagire con la Knowledge Base, mediante domande, senza la necessità di conoscere la complessa sintassi Prolog. Le regole sfruttano i fatti presenti nella KB per dedurre e fornire all'utente le informazioni da lui richieste. Sono stati previsti due task principali:

1. Trova film in base a determinate caratteristiche
2. Trova la piattaforma migliore per i film selezionati

```

Inserisci un valore (1-4): 2

--- Caricamento della KB... Attendi... ---
Avvio del motore Prolog e caricamento della Knowledge Base...

=== CINELOGIC: KNOWLEDGE BASE ===
1) Trova film in base alle tue preferenze
2) Trova la migliore piattaforma di streaming per i film trovati
3) Esci
Inserisci un valore (1-3):

```

3.4.1. Trova film in base a determinate caratteristiche

Il primo **task** consiste nell'ottenere tutti i film che soddisfano determinate caratteristiche indicate dall'utente.

Per fare ciò, sono state create una serie di regole corrispondenti a tutte le possibili scelte possibili da parte dell'utente.

Si presentano alcuni esempi presenti nel file "KnowledgeBase.py":

```

# Anno di uscita
"film_recente(ID) :- release_year(ID, Uscita), Uscita > 2010.\n",
"film_tra_2000_2010(ID) :- release_year(ID, Uscita), Uscita >= 2000, Uscita <= 2010.\n",
"film_pre_2000(ID) :- release_year(ID, Uscita), Uscita < 2000, Uscita > 0.\n\n",

```

Queste regole segmentano i film in base all'anno di uscita, distinguendo tra produzioni **pre-2000**, produzioni del decennio **2000-2010** e film considerati **recenti** (post-2010).

```

# Durata del film
"film_breve_durata(ID) :- runtime(ID, Durata), Durata <= 60, Durata > 0.\n",
"film_media_durata(ID) :- runtime(ID, Durata), Durata > 60, Durata <= 90.\n",
"film_lunga_durata(ID) :- runtime(ID, Durata), Durata > 90.\n\n",

```

```

# Categorizzazione durata
"film_durata(ID, breve) :- film_breve_durata(ID).\n",
"film_durata(ID, media) :- film_media_durata(ID).\n",
"film_durata(ID, lunga) :- film_lunga_durata(ID).\n\n",

```

La regola traduce il valore numerico del runtime in categorie discrete: un film è considerato di **breve durata** sotto i 60 minuti, **media** tra 60 e 90, e di **lunga durata** oltre i 90 minuti.

Esempio di esecuzione:

=== CINELOGIC: KNOWLEDGE BASE ===

1) Trova film in base alle tue preferenze

2) Trova la migliore piattaforma di streaming per i film trovati

3) Esci

Inserisci un valore (1-3): 1

Periodo di uscita:

1) Recente (>2010)

2) Tra 2000 e 2010

3) Pre 2000

Scegli:

Generi disponibili:

1) Western

2) Scifi

3) Romance

4) Drama

5) Horror

6) Thriller

7) Comedy

8) Crime

9) Documentation

10) Family

11) Action

12) Fantasy

13) Animation

14) Music

15) History

16) War

17) European

18) Sport

19) Reality

Scegli il numero del genere:

Durata:

- 1) Breve (<60 min)
- 2) Media (60-90 min)
- 3) Lunga (>90 min)

Scegli:

```
Ricerca nella Knowledge Base in corso...
```

```
=== FILM TROVATI ===
```

- My Normal (ID: 100157)
- Open Five (ID: 104156)
- Slackers (ID: 107917)
- Mambo Italiano (ID: 132528)
- Crazy For Christmas (ID: 159890)
- Yours, Mine & Ours (ID: 165227)
- The Engagement Ring (ID: 171382)
- Confessions Of An American Bride (ID: 178417)
- Greener Mountains (ID: 200876)
- It Had To Be You (ID: 24154)
- Last Resort (ID: 26829)
- Playing House (ID: 28493)
- Last Chance Cafe (ID: 320619)
- Mulligans (ID: 32785)
- Loving Annabelle (ID: 33399)
- Jack And Jill Vs. The World (ID: 55859)

3.4.2. Sulla base delle preferenze dell'utente, trova la piattaforma migliore per i film selezionati

Il secondo task consiste nell'individuare la piattaforma di streaming ottimale da consigliare all'utente.

Per fare ciò, il sistema recupera gli ID dei film target individuati durante il primo task. Inoltre, sono state definite nuove regole Prolog (prezzo_economy, prezzo_medio, prezzo_costoso) per classificare il costo di sottoscrizione mensile delle varie piattaforme.

È stata implementata una funzione in Python denominata `find_best_streaming_platform` la quale interroga la Knowledge Base passando i film target e applicando i filtri di budget scelti dall'utente. La miglior piattaforma restituita sarà dunque quella che, sulla base del prezzo di sottoscrizione indicato dall'utente, contiene il maggior numero di film target, massimizzando così il rapporto tra la spesa sostenuta e la disponibilità di contenuti in linea con i gusti personali dell'utente.

```
def find_best_streaming_platform(prolog, film_ids):
    price_filters = []

    # Selezione del prezzo
    while not price_filters:
        try:
            price_input = int(input("\nSeleziona il tuo budget per lo streaming:\n"
                                    "1) Economica (< 9.99)\n2) Media (fino a 9.99)\n3) Costosa (qualsiasi prezzo)\nScegli: "))

            if price_input == 1:
                price_filters = ["prezzo_economy"]
            elif price_input == 2:
                price_filters = ["prezzo_economy", "prezzo_medio"]
            elif price_input == 3:
                price_filters = ["prezzo_economy", "prezzo_medio", "prezzo_costoso"]
            else:
                print("Input non valido.")
        except ValueError:
            print("Inserisci un numero.")

    platform_count = {}
```

```
print("\nAnalisi delle piattaforme in corso...")
for film_id in film_ids:
    # Per ogni film, cerco su che piattaforma si trova
    query_piattaforma = f"streaming_service({film_id}, Piattaforma)"

    try:
        results = list(prolog.query(query_piattaforma))
        for result in results:
            piattaforma = result["Piattaforma"]

            # Controllo se il film rispetta almeno uno dei filtri di prezzo
            price_match = False
            for price_filter in price_filters:
                check_query = f"{price_filter}({film_id})"
                # Se la lista tornata non è vuota, il fatto è vero
                if list(prolog.query(check_query)):
                    price_match = True
                    break

            if price_match:
                platform_count[piattaforma] = platform_count.get(piattaforma, 0) + 1

    except Exception as e:
        print(f"Errore query per ID {film_id}: {e}")
```

```
# Risultato finale
if platform_count:
    best_platform = max(platform_count, key=platform_count.get)
    print("\n=== RISULTATO ===")
    print(f"La piattaforma più consigliata per te è: **{best_platform.upper()}**")
    print(
        f"Contiene {platform_count[best_platform]} dei film che hai cercato ad un prezzo compatibile col tuo budget.")
else:
    print("\nNessuna piattaforma offre questi film al prezzo selezionato.")
```

Di seguito le regole per la definizione del prezzo di sottoscrizione:

```
rules = [
    # Prezzo di sottoscrizione
    "prezzo_economy(ID) :- monthly_subscription_cost(ID, Cost), Cost < 9.99.\n",
    "prezzo_medio(ID) :- monthly_subscription_cost(ID, Cost), Cost := 9.99.\n",
    "prezzo_costoso(ID) :- monthly_subscription_cost(ID, Cost), Cost > 9.99.\n\n",
```

La regola traduce il valore numerico del runtime in categorie discrete: un film è considerato di **breve durata** sotto i 60 minuti, **media** tra 60 e 90, e di **lunga durata** oltre i 90 minuti.

Di seguito un esempio di esecuzione:

```
=== CINELOGIC: KNOWLEDGE BASE ===
1) Trova film in base alle tue preferenze
2) Trova la migliore piattaforma di streaming per i film trovati
3) Esci
Inserisci un valore (1-3): 2

Seleziona il tuo budget per lo streaming:
1) Economica (< 9.99)
2) Media (fino a 9.99)
3) Costosa (qualsiasi prezzo)
Scegli: 1

Analisi delle piattaforme in corso...

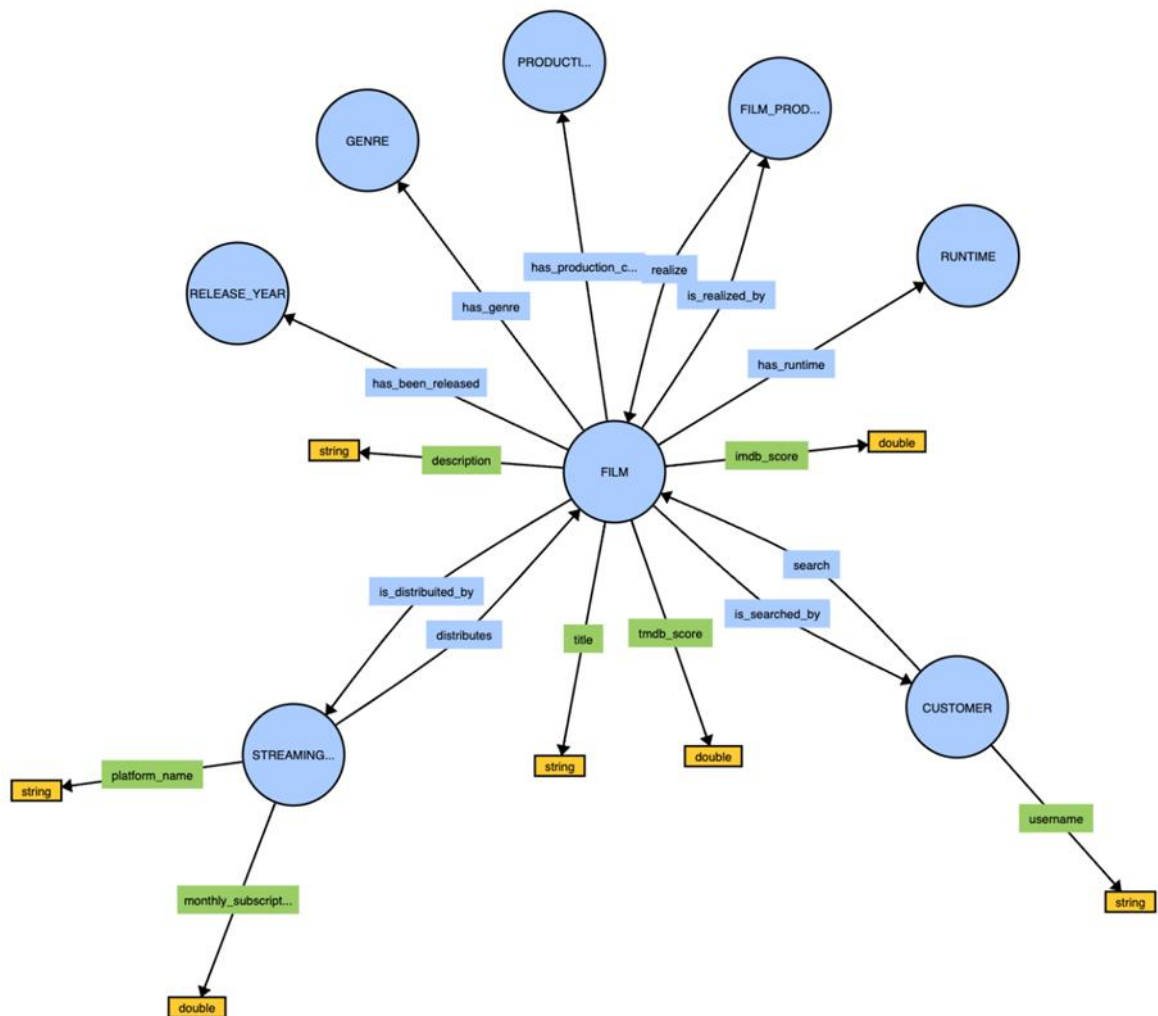
=== RISULTATO ===
La piattaforma più consigliata per te è: **DISNEY**
Contiene 4 dei film che hai cercato ad un prezzo compatibile col tuo budget.
```

4. Ontologia

4.1. Sommario

In informatica, un'ontologia è un sistema di rappresentazione formale utilizzato per descrivere la realtà e strutturare la conoscenza. Essa agisce come un modello semantico che permette di definire le entità (oggetti, concetti, ecc.) e le relazioni che intercorrono tra di esse all'interno di un determinato dominio di interesse.

Per il nostro progetto, abbiamo scelto di sviluppare un'ontologia dedicata all'ecosistema dei servizi di streaming cinematografico online. Questo strumento consente agli utenti di esplorare e confrontare le principali piattaforme di streaming (come Amazon, Netflix, Hulu, Disney+, ecc.), analizzando nel dettaglio informazioni semantiche quali i generi disponibili, le annate di uscita dei titoli, gli studi di produzione e i relativi costi di abbonamento.



4.2. Protégé

Per la realizzazione pratica dell'ontologia, abbiamo impiegato l'editor open source Protégé, uno standard nel campo del Web Semantico. L'architettura dell'ontologia prevede la definizione di una gerarchia di classi: la maggior parte di esse rappresenta i metadati e le categorie che descrivono un film nel dettaglio (es. Release_year, Genre, Streaming_service).

Sono inoltre presenti classi che modellano gli "attori" (o stakeholder) che interagiscono attivamente in questo mercato, come i clienti (Customer) e gli studi di produzione (Film_production_studios).

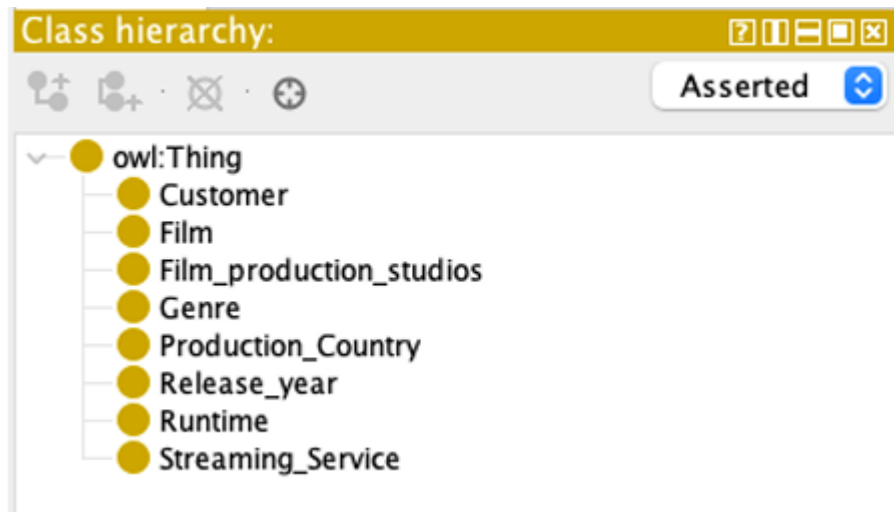


Figura 4-1: Classi dell'Ontologia

Dopo aver modellato la tassonomia delle classi, l'ontologia è stata arricchita definendo una serie di proprietà semantiche, specificatamente suddivise in Object Properties e Data Properties:

- Le Object Properties (Proprietà d'Oggetto) servono a instaurare relazioni logiche per connettere due individui, appartenenti alla stessa classe o a classi differenti (ad esempio, la relazione tra un film e la piattaforma che lo distribuisce).
- Le Data Properties (Proprietà dei Dati) collegano invece un individuo a un tipo di dato primitivo, come una stringa testuale, un valore booleano o un numero (ad esempio, il titolo esatto del film o il costo in formato numerico).

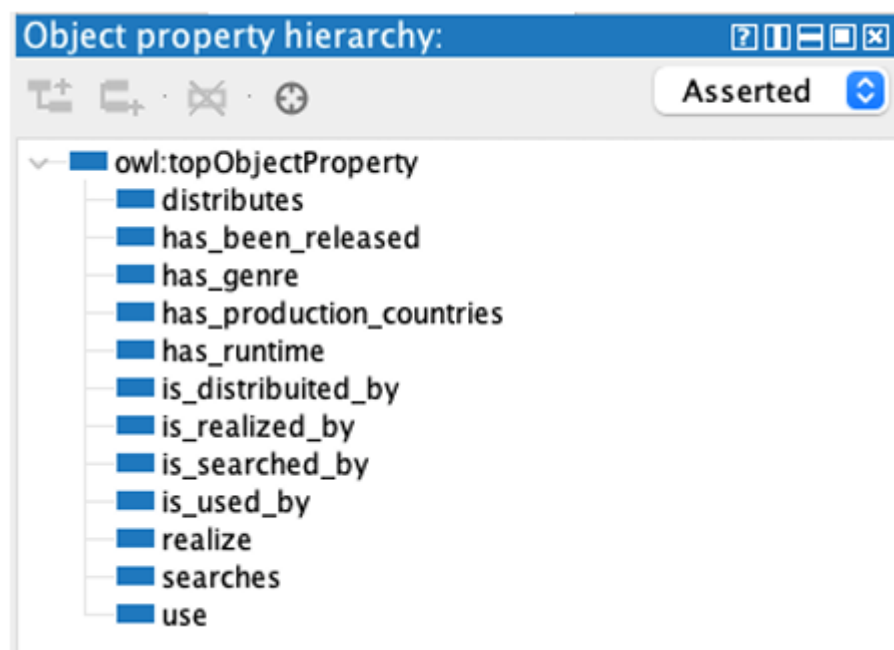


Figura 4-2: Object-Properties

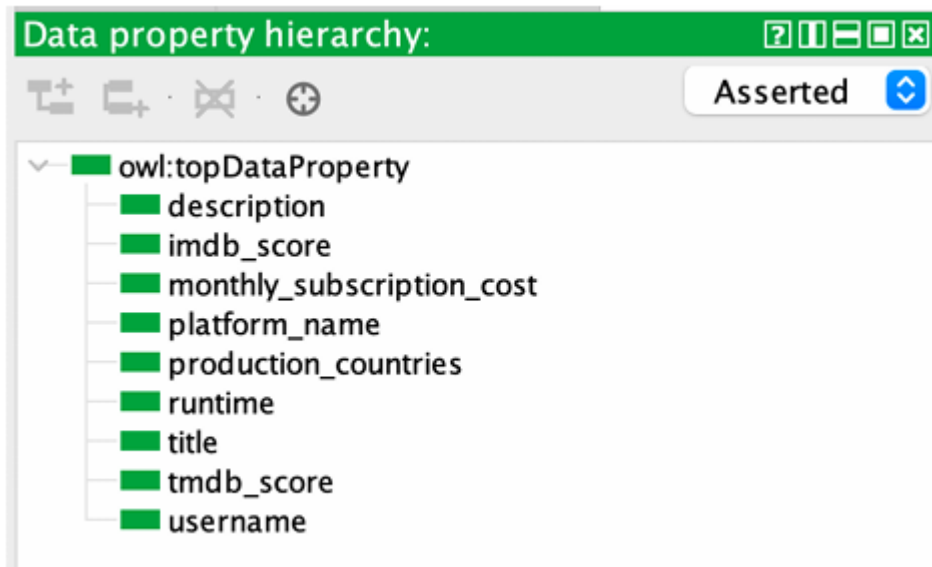


Figura 4-3: Data-Properties

Successivamente siamo passati alla realizzazione degli individui stessi che rappresentano alcuni esempi di film presenti nel dataset:



Figura 4-4: Individui della classe Film



Figura 4-5: Individui della classe Genre

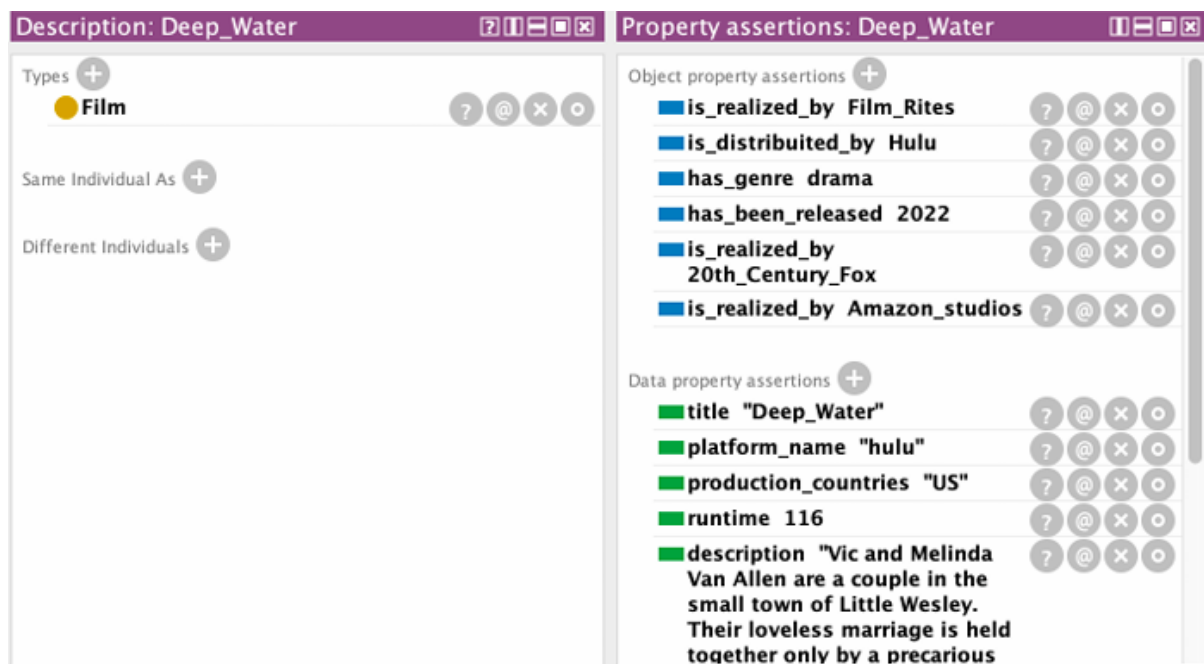


Figura 4-6: Esempio di individuo con properties annesse

Possiamo anche interrogare l'ontologia tramite l'utilizzo di due tipologie differenti di query, DL query e SPARQL query.

SPARQL query:		
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> PREFIX owl: <http://www.w3.org/2002/07/owl#> PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> PREFIX xsd: <http://www.w3.org/2001/XMLSchema#> PREFIX O: <http://www.semanticweb.org/gabrielepaladini/ontologies/2024/6/untitled-ontology-7#>		
SELECT ?Film ?production_countries ?runtime WHERE { ?Film O:production_countries ?production_countries. ?Film O:runtime ?runtime. }		
Film	production_countries	runtime
Deep_Water	"US"	"116"^^<http://www.w3.org/2001/XMLSchema#decimal>
The_Brotherhood	"US"	"84"^^<http://www.w3.org/2001/XMLSchema#decimal>
Black_Island	"DE"	"105"^^<http://www.w3.org/2001/XMLSchema#decimal>
Ready_Player_One	"US"	"140"^^<http://www.w3.org/2001/XMLSchema#decimal>
Escape_from_Planet_Earth	"CA"	"89"^^<http://www.w3.org/2001/XMLSchema#decimal>
D-Day_The_Price_Of_Freedom	"US"	"53"^^<http://www.w3.org/2001/XMLSchema#decimal>

Figura 4-7: SPARQL query con visualizzazione dei Film, Production_countries e runtime

SPARQL query:	
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> PREFIX owl: <http://www.w3.org/2002/07/owl#> PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> PREFIX xsd: <http://www.w3.org/2001/XMLSchema#> PREFIX O: <http://www.semanticweb.org/gabrielepaladini/ontologies/2024/6/untitled-ontology-7#>	
SELECT ?Film ?is_distributed_by WHERE { ?Film O:is_distributed_by ?is_distributed_by. ?Film O:imdb_score 5.8 }	
Film	is_distributed_by
Escape_from_Planet_Earth	Netflix
Deep_Water	Hulu

Figura 4-8: SPARQL query con visualizzazione dei Film che hanno come Object-properties "is_distributed_by" e un "imdb_score" pari a 5.8

DL query:

Query (class expression)

Film that has_genre value scifi and has_been_released value 2018

Execute

Add to ontology

Query results

Instances (1 of 1)

Ready_Player_One

?

Query for

☐ Direct superclasses
 ☐ Superclasses
 ☐ Equivalent classes

Figura 4-9: DL-query con ricerca di film che hanno come genere "scifi" e come anno di rilascio "2018"

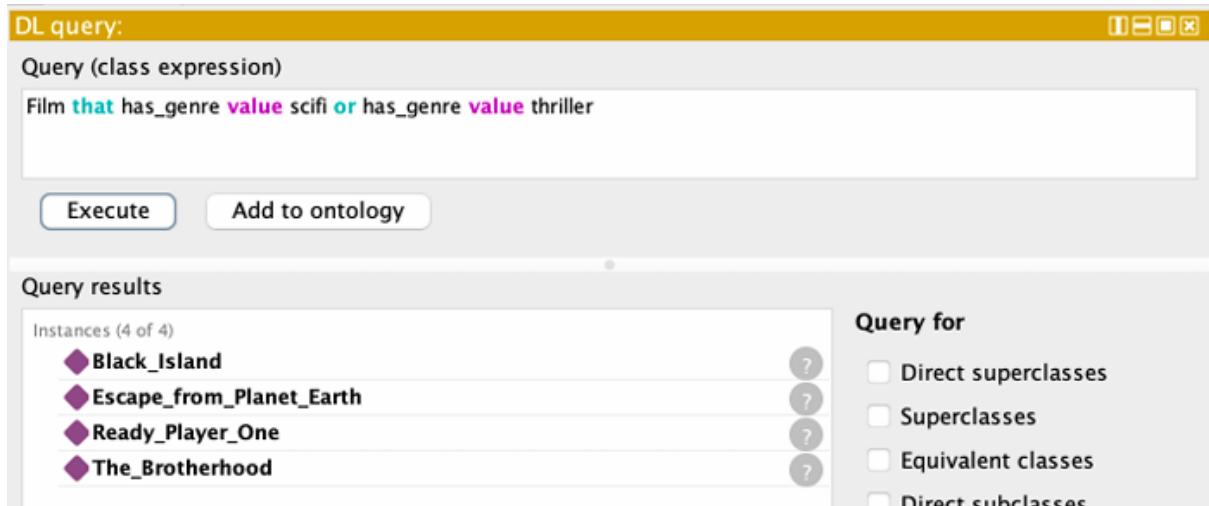


Figura 4-10:DL-query con ricerca di film che hanno come genere "scifi" oppure "thriller"

4.3. Owlready2

Per l'integrazione e l'interrogazione dell'ontologia all'interno del nostro applicativo Python, abbiamo impiegato la libreria owlready2. Questo strumento ci ha consentito di caricare il modello semantico, precedentemente generato tramite l'editor Protégé, e di esplorarne la struttura logica e le istanze in modo estremamente efficiente.

Durante l'esecuzione del modulo dedicato, il programma presenta all'utente un'interfaccia interattiva a terminale da cui è possibile navigare all'interno del dominio di conoscenza. Nello specifico, l'utente può scegliere di:

- Visualizzare la tassonomia delle Classi.
- Esplorare le Object-Properties (Proprietà d'Oggetto) e le Data Properties (Proprietà dei Dati)
- Eseguire una serie di query di esempio per testare le capacità di ricerca avanzata del sistema

Di seguito viene riportato il menù iniziale, che funge da punto di accesso per l'esplorazione dell'ontologia e dal quale l'utente può effettuare le proprie scelte operative:

```
--- Avvio Ontologia ---

=== BENVENUTO NELL'ONTOLOGIA DI CINELOGIC ===
Caricamento ontologia in corso... Attendi...
Ontologia caricata con successo!

Seleziona un'operazione:
1) Visualizzazione Classi
2) Visualizzazione Proprietà d'Oggetto
3) Visualizzazione Proprietà dei Dati
4) Esegui query specifiche
5) Esci dall'Ontologia

Inserisci un valore (1-5): |
```

Selezionando la prima opzione, si potranno visualizzare le classi presenti all'interno dell'ontologia:

```
--- CLASSI PRESENTI NELL'ONTOLOGIA ---  
- Ontologia.owx.Customer  
- Ontologia.owx.Film  
- Ontologia.owx.Film_production_studios  
- Ontologia.owx.Genre  
- Ontologia.owx.Release_year  
- Ontologia.owx.Streaming_Service
```

Vorresti esplorare le istanze di una delle seguenti classi?

- 1) Film
- 2) Release_year
- 3) Streaming_service
- 4) Genre
- 5) Customer
- 6) Film_production_studios
- 7) Torna al menu principale

Inserisci un valore (1-7):

Possiamo, inoltre, esaminare da vicino le classi selezionando quella di interesse tra quelle visualizzate a schermo.

- 1) Selezionando la prima classe, verrà mostrato a schermo la lista dei film presenti:

```
LISTA FILM PRESENTI:  
- Ontologia.owx.Film  
- Ontologia.owx.Black_Island  
- Ontologia.owx.Deep_Water  
- Ontologia.owx.Escape_from_Planet_Earth  
- Ontologia.owx.Ready_Player_One  
- Ontologia.owx.The_Brotherhood  
- Ontologia.owx.D-Day: The Price Of Freedom
```

- 2) Selezionando la seconda classe, verrà mostrato a schermo un elenco degli anni di rilascio dei film:

LISTA ANNI DI RILASCIO PRESENTI:

- `Ontologia.owx.Release_year`
- `Ontologia.owx.2005`
- `Ontologia.owx.2012`
- `Ontologia.owx.2016`
- `Ontologia.owx.2017`
- `Ontologia.owx.2018`
- `Ontologia.owx.2021`
- `Ontologia.owx.2022`
- `Ontologia.owx.2023`
- `Ontologia.owx.2024`

- 3) Selezionando la terza classe, verrà mostrato a schermo un elenco contenente tutti i servizi di streaming presenti all'interno della nostra ontologia:

LISTA DEI SERVIZI DI STREAMING PRESENTI:

- `Ontologia.owx.Streaming_Service`
- `Ontologia.owx.Amazon`
- `Ontologia.owx.Hbo`
- `Ontologia.owx.Hulu`
- `Ontologia.owx.Netflix`
- `Ontologia.owx.Disney+`

- 4) Selezionando la quarta classe, verrà mostrato a schermo l'elenco dei generi presenti:

LISTA DEI GENERI PRESENTI:

- `Ontologia.owx.Genre`
- `Ontologia.owx.action`
- `Ontologia.owx.comedy`
- `Ontologia.owx.crime`
- `Ontologia.owx.documentation`
- `Ontologia.owx.drama`
- `Ontologia.owx.fantasy`
- `Ontologia.owx.horror`
- `Ontologia.owx.scifi`
- `Ontologia.owx.thriller`
- `Ontologia.owx.western`

- 5) Selezionando la quinta classe, verrà mostrato a schermo l'elenco dei clienti presenti:

LISTA DEI CLIENTI PRESENTI:

- `Ontologia.owx.Customer`
- `Ontologia.owx.CinephileAlex`
- `Ontologia.owx.FlickFanFabio`
- `Ontologia.owx.MovieBuffMarco`
- `Ontologia.owx.SeriesFanAnna`

- 6) Selezionando la sesta classe, verrà mostrato a schermo l'elenco degli studi di produzione presenti:

LISTA DEGLI STUDI DI PRODUZIONE PRESENTI:

- `Ontologia.owx.Film_production_studios`
- `Ontologia.owx.Amazon_studios`
- `Ontologia.owx.Amblin_Entertainment`
- `Ontologia.owx.DeLine_Pictures`
- `Ontologia.owx.Film_Rites`
- `Ontologia.owx.Netflix_Studios`
- `Ontologia.owx.RainMaker_Entertainment`
- `Ontologia.owx.20th_Century_Fox`

- 7) Verrà richiesto alla fine dell'analisi di ciascuna delle classi proposte se l'utente desidera esaminare un'altra classe oppure no:

Vorresti esplorare le istanze di una delle seguenti classi?

- 1) `Film`
- 2) `Release_year`
- 3) `Streaming_service`
- 4) `Genre`
- 5) `Customer`
- 6) `Film_production_studios`
- 7) `Torna al menu principale`

Selezionando la seconda opzione, si avrà la possibilità di visualizzare le 'proprietà d'oggetto' contenute all'interno dell'ontologia:

PROPRIETÀ D'OGGETTO PRESENTI NELL'ONTOLOGIA:

- `Ontologia.owx.distributes`
- `Ontologia.owx.has_been_released`
- `Ontologia.owx.has_genre`
- `Ontologia.owx.has_production_countries`
- `Ontologia.owx.has_runtime`
- `Ontologia.owx.is_distributed_by`
- `Ontologia.owx.is_realized_by`
- `Ontologia.owx.is_searched_by`
- `Ontologia.owx.is_used_by`
- `Ontologia.owx.realize`
- `Ontologia.owx.searches`
- `Ontologia.owx.use`

Selezionando la terza opzione, si avrà la possibilità di esaminare le 'proprietà dei dati' presenti nell'ontologia:

PROPRIETÀ DEI DATI PRESENTI NELL'ONTOLOGIA:

- `Ontologia.owx.description`
- `Ontologia.owx.imdb_quotes`
- `Ontologia.owx.imdb_score`
- `Ontologia.owx.imdb_votes`
- `Ontologia.owx.monthly_subscription_cost`
- `Ontologia.owx.platform_name`
- `Ontologia.owx.production_countries`
- `Ontologia.owx.runtime`
- `Ontologia.owx.title`
- `Ontologia.owx.tmdb_popularity`
- `Ontologia.owx.tmdb_quotes`
- `Ontologia.owx.tmdb_score`
- `Ontologia.owx.username`

Selezionando la quarta operazione, si potranno selezionare ed eseguire alcune query di esempio:

```
Scegli una query da eseguire:  
1) Lista film presenti su 'Amazon'  
2) Lista film di genere 'Sci-Fi'  
3) Studi di produzione del film 'Deep Water'  
4) Torna al menu principale
```

```
Inserisci un valore: 1  
FILM PRESENTI SU AMAZON:  
- Ontologia.owx.The_Brotherhood  
- Ontologia.owx.D-Day:_The_Price_Of_Freedom
```

Figura 4-11: Esecuzione della Prima Query

```
Inserisci un valore: 2  
FILM DI GENERE SCI-FI:  
- Ontologia.owx.Escape_from_Planet_Earth  
- Ontologia.owx.Ready_Player_One  
- Ontologia.owx.The_Brotherhood
```

Figura 4-12: Esecuzione della Seconda Query

```
Inserisci un valore: 3  
STUDI DI PRODUZIONE DEL FILM 'DEEP WATER':  
- Ontologia.owx.Amazon_studios  
- Ontologia.owx.Film_Rites  
- Ontologia.owx.20th_Century_Fox
```

Figura 4-13: Esecuzione della Terza Query

5. Conclusioni

Il Progetto CineLogic rappresenta una utile architettura per l'esplorazione e la raccomandazione di contenuti cinematografici. Sono state usate diverse tecniche di *Machine Learning* supervisionato (algoritmo KNN ottimizzato), *Natural Language Processing* (tecnica TF-IDF e Similarità del Coseno per il Content-Based Filtering), Sistemi Esperti (Knowledge Base in Prolog) e Web Semantico (Ontologie OWL) ha permesso di creare un applicativo robusto e multidimensionale.

Tuttavia, vi sono ampi margini di miglioramento ed estensione del progetto. Un'evoluzione naturale consisterebbe nell'ampliamento del dataset attuale, integrando basi di dati più massicce provenienti da ulteriori piattaforme di streaming. La combinazione di questi dataset consentirebbe di arricchire il sistema con una gamma più vasta di film: questo non solo aumenterebbe la varietà delle raccomandazioni testuali, ma offrirebbe al modello di classificazione un bacino di addestramento più bilanciato per le classi minoritarie.

Inoltre, sviluppi futuri potrebbero prevedere l'implementazione di un'interfaccia grafica (GUI, ad esempio tramite framework web) per rendere l'interazione con la Knowledge Base e l'Ontologia visivamente più intuitiva per l'utente finale.

6. Bibliografia

Materiale Didattico e Testi di Riferimento

1. Materiale didattico del corso di ingegneria della conoscenza presenti su e-learning

Documentazione Software e Librerie

1. Scikit-learn developers. Scikit-learn: Machine Learning in Python. Documentazione ufficiale per i moduli KNeighborsClassifier, RandomizedSearchCV, TfidfVectorizer e le metriche di similarità (cosine_similarity). Reperibile all'indirizzo: <https://scikit-learn.org/>
2. The pandas development team. Pandas Documentation. Documentazione ufficiale per l'elaborazione e la pulizia dei dati (metodi pd.read_csv, pd.cut, DataFrame). Reperibile all'indirizzo: <https://pandas.pydata.org/>
3. Lamy, J. B. (2017). Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. Artificial Intelligence in Medicine. Documentazione ufficiale della libreria owlready2. Reperibile all'indirizzo: <https://owlready2.readthedocs.io/>
4. Stanford Center for Biomedical Informatics Research. Protégé. Piattaforma open-source per la modellazione di ontologie. Reperibile all'indirizzo: <https://protege.stanford.edu/>
5. SWI-Prolog e PySWIP. Documentazione per il motore logico e le interfacce bridge per Python. Reperibili agli indirizzi: <https://www.swi-prolog.org/> e <https://pypi.org/project/pyswip/>